

Global Academy of Technology

RR Nagar, Bengaluru – 560098

Department of Computer Science and Engineering

Accredited by NBA 2019-2022)



I Semester

C Programming

21CSE13

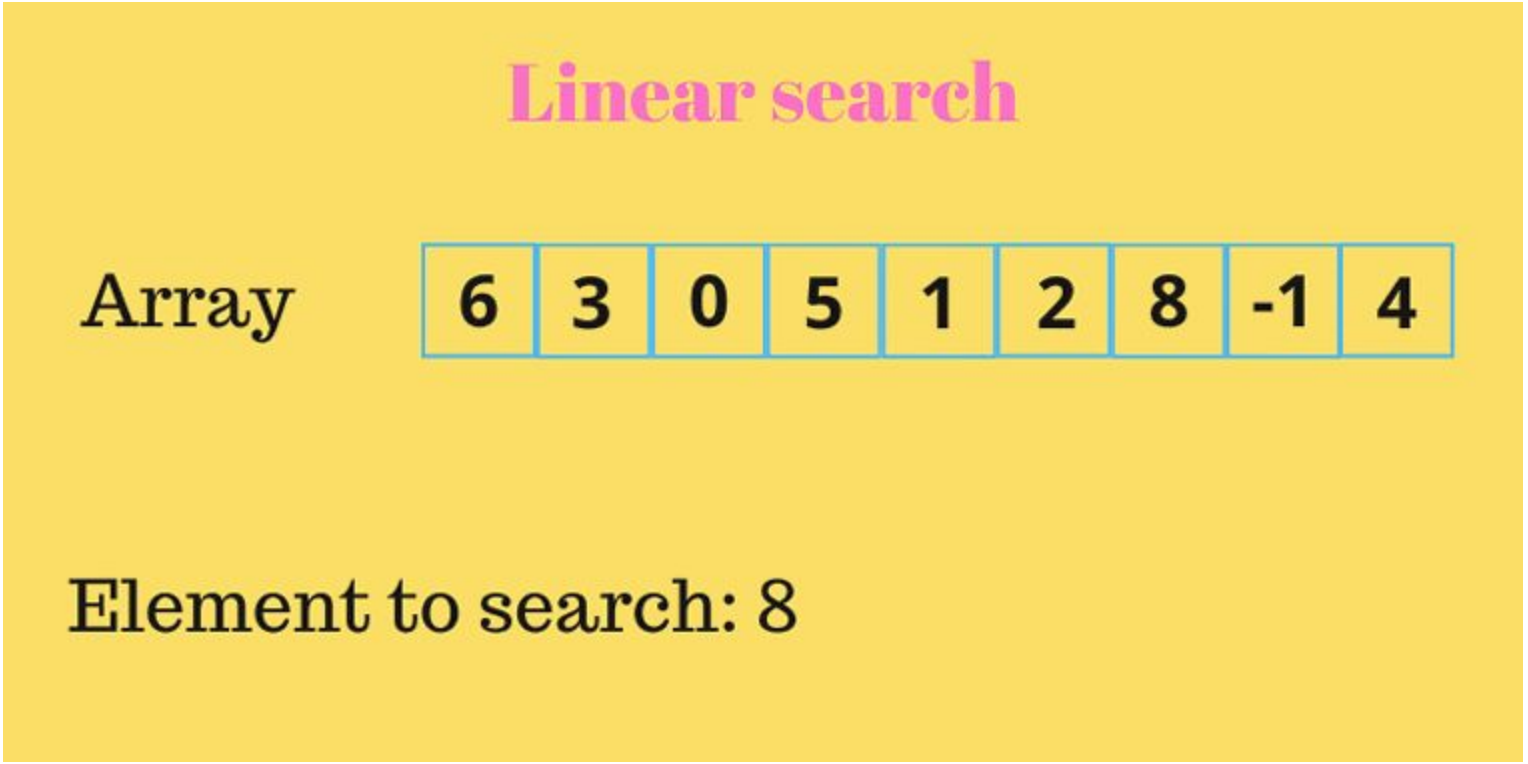
Course Instructor
Mrs. Vanishree M L
Asst. Professor
CSE, GAT

Module 3

Arrays and Strings

1-d Array Examples

4. Write a program to search for a number in an array using Linear search.



1-d Array Examples

Write a program to search for a number in an array using Linear search.

```
#include<stdio.h>
void main()
{
    int a[21], n, num, i, count=0;
    printf("Enter the size of the array : ");
    scanf("%d", &n);
    printf("Enter array elements\n");
    for(i=0;i<n;i++)
    scanf("%d", &a[i]);
    printf("Enter a number to be searched : ");
    scanf("%d", &num);
```

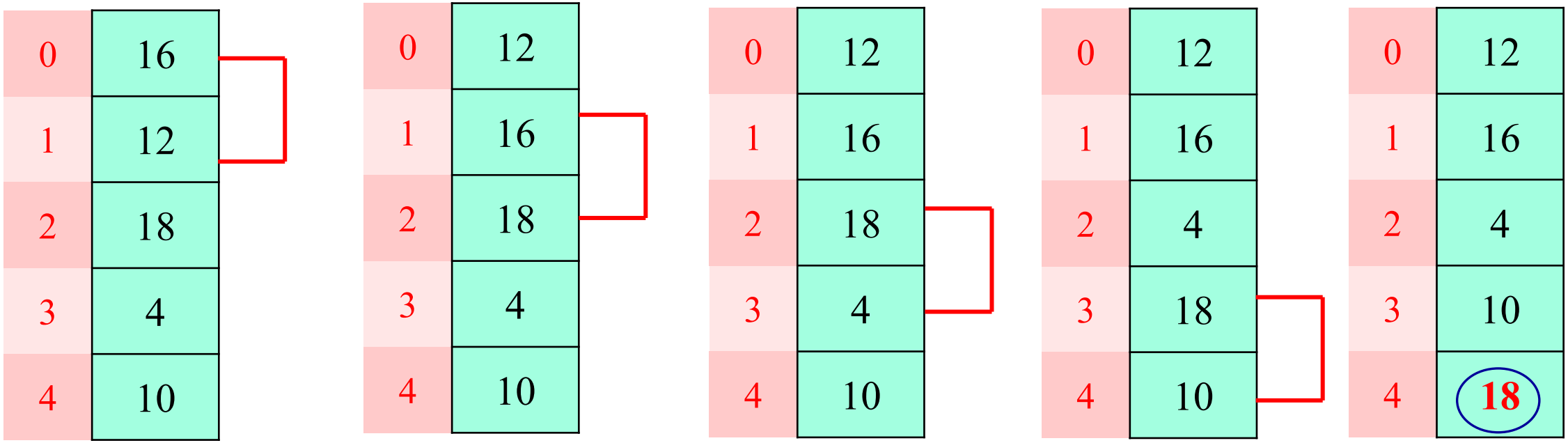
```
 for(i=0;i<n;i++)
     {
         if(num==a[i])
         count++;
     }
     if(count==0)
     printf("Element not found");
     else
     printf("Element found %d times", count);
}
```

Write a C program to sort the elements using Bubble sort method.

By default, ascending order

1st PASS

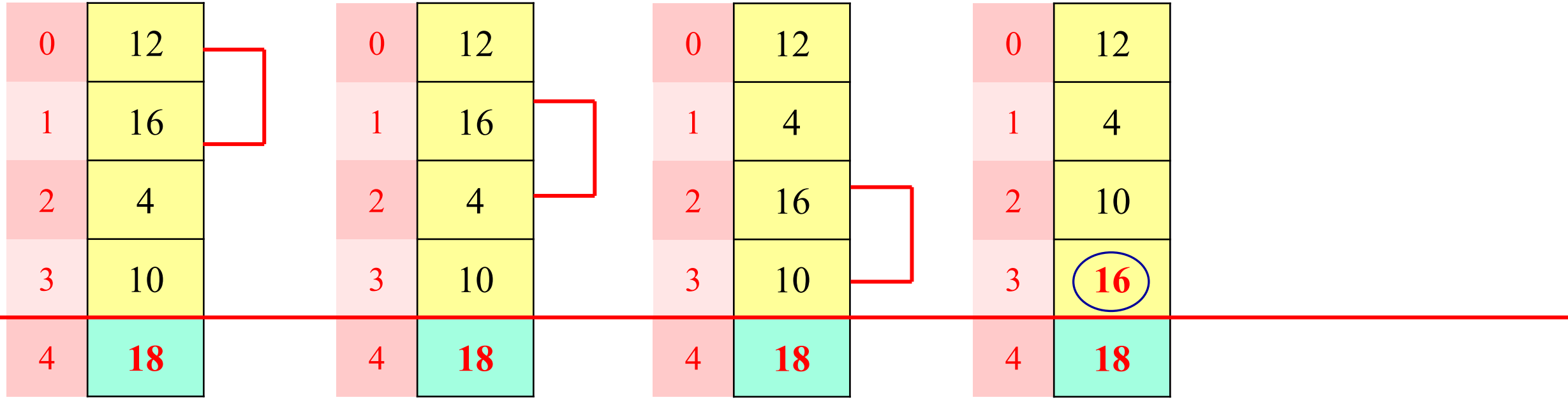
4 Comparisons



Write a C program to sort the elements using Bubble sort method.

2nd PASS

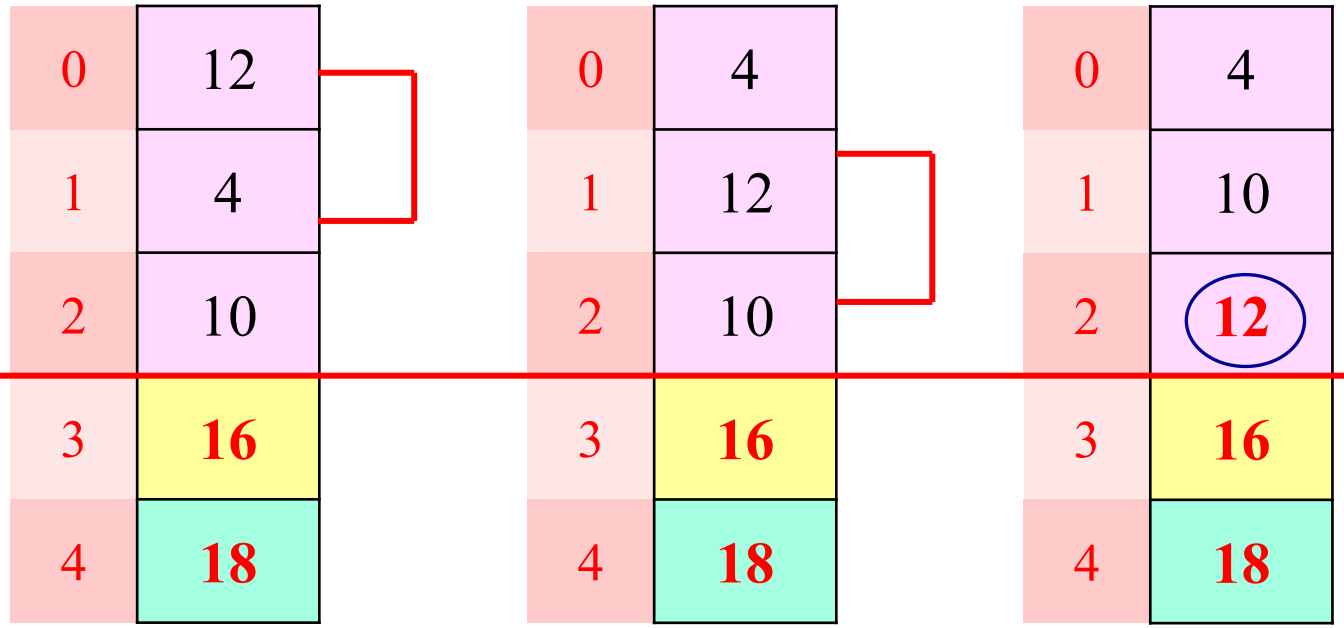
3 Comparisons



Write a C program to sort the elements using Bubble sort method.

3rd PASS

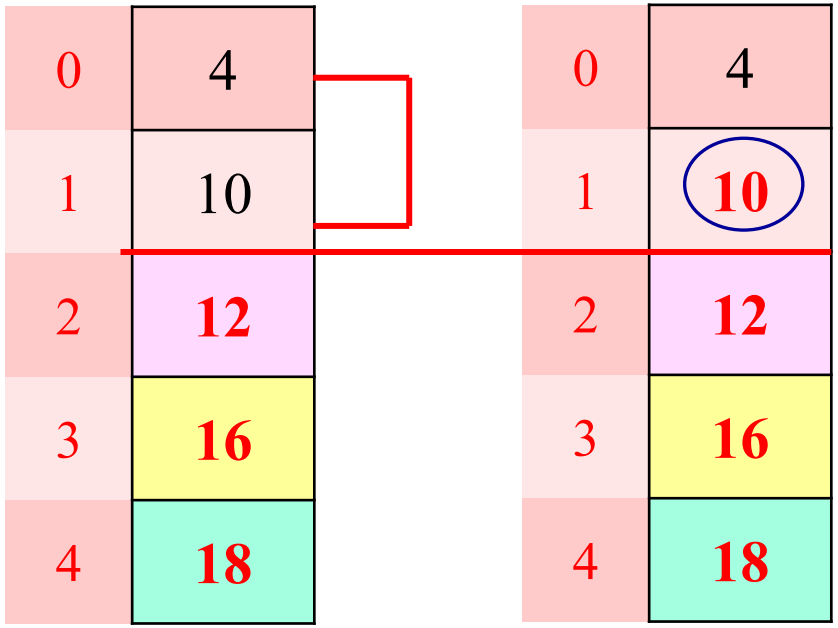
2 Comparisons



Write a C program to sort the elements using Bubble sort method.

4th PASS

1 Comparison



Sorted elements in ascending order

0	4
1	10
2	12
3	16
4	18

Write a C program to sort the elements using Bubble sort method.

```
#include<stdio.h>
void main()
{
    int n,i,j,temp,a[10];
    printf("Enter the number of elements :");
    scanf("%d",&n);
    printf("Enter the elements to sort\n");
    for(i=0;i<n;i++)
    scanf("%d",&a[i]);
    printf("Given array:\n");
    for(i=0;i<n;i++)
    printf("%d\n",a[i]);
```

```
    for(i=0;i<n-1;i++)  // no. of passes
    {
        for(j=0;j<n-1-i;j++)  // no. of comparisons
        {
            if(a[j]>a[j+1])
            {
                temp=a[j];
                a[j]=a[j+1];
                a[j+1]=temp;
            }
        }
    }
    printf("Sorted array:\n");
    for(i=0;i<n;i++)
    printf("%d\n",a[i]);
}
```

No. of elements n = 5

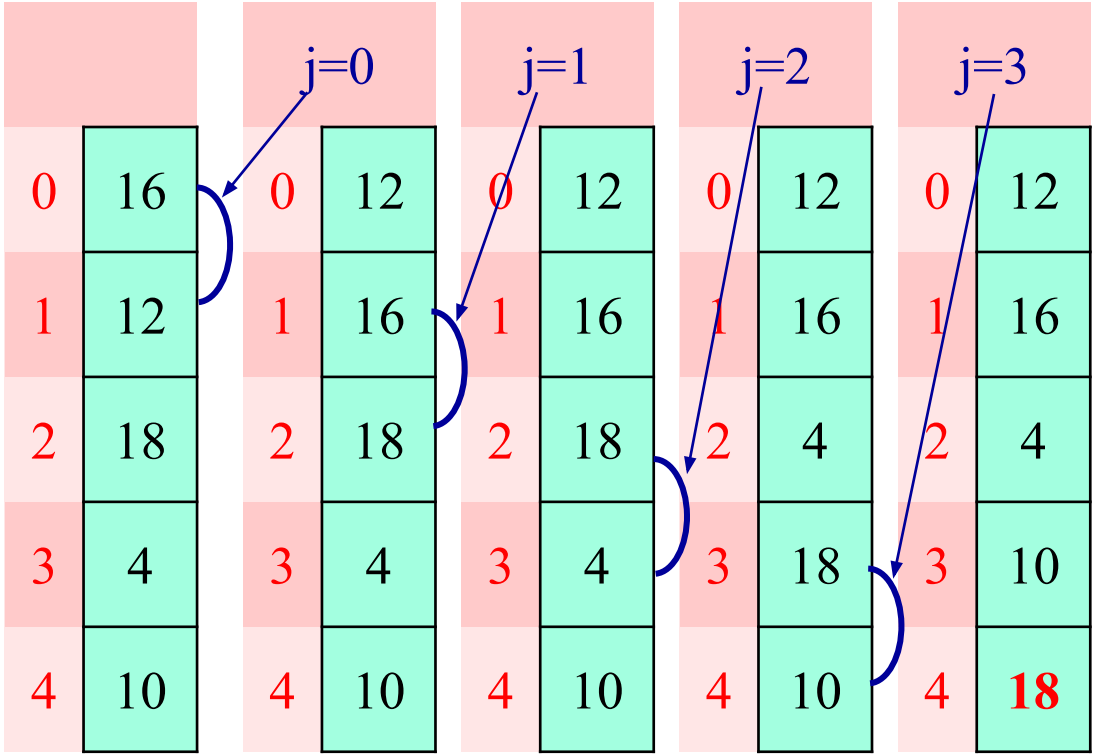
(1) **i = 0** i < n – 1 (0 < 4) True

(a) **j = 0** j < n – 1 – i (0 < 4) True
Compare a[j] > a[j+1] a[0] > a[1] True
Swap them a[0] = 12, a[1] = 16

(b) **j = 1** j < n – 1 – i (1 < 4) True
Compare a[j] > a[j+1] a[1] > a[2] False

(c) **j = 2** j < n – 1 – i (2 < 4) True
Compare a[j] > a[j+1] a[2] > a[3] True
Swap them a[2] = 4, a[3] = 18

(d) **j = 3** j < n – 1 – i (3 < 4) True
Compare a[j] > a[j+1] a[3] > a[4] True
Swap them a[3] = 10, a[4] = 18



No. of elements n = 5

(2) i = 1 i < n – 1 (1 < 4) True

(a) j = 0 j < n – 1 – i (0 < 3) True
Compare a[j] > a[j+1] a[0] > a[1] False

(b) j = 1 j < n – 1 – i (1 < 3) True
Compare a[j] > a[j+1] a[1] > a[2] True
Swap them a[1] = 4, a[2] = 16

(c) j = 2 j < n – 1 – i (2 < 3) True
Compare a[j] > a[j+1] a[2] > a[3] True
Swap them a[2] = 10, a[3] = 16

		j=0		j=1		j=2	
0	12	0	12	0	12	0	12
1	16	1	16	1	4	1	4
2	4	2	4	2	16	2	10
3	10	3	10	3	10	3	**16**
4	**18**	4	**18**	4	**18**	4	**18**

No. of elements n = 5

(3) i = 2 i < n – 1 (2< 4) True

(a) j = 0 j < n – 1 – i (0 < 2) True
Compare a[j] > a[j+1] a[0] > a[1] True
Swap them a[0] = 4, a[1] = 12

(b) j = 1 j < n – 1 – i (1 < 2) True
Compare a[j] > a[j+1] a[1] > a[2] True
Swap them a[1] = 10, a[2] = 12

		j=0		j=1	
0	12	0	4	0	4
1	4	1	12	1	10
2	10	2	10	2	**12**
3	**16**	3	**16**	3	**16**
4	**18**	4	**18**	4	**18**

No. of elements n = 5

(4) i = 3 i < n – 1 (3< 4) True

(a) j = 0 j < n – 1 – i (0 < 1) True

Compare a[j] > a[j+1] a[0] > a[1] False

0	4
1	10
2	**12**
3	**16**
4	**18**

	j=0
0	4
1	**10**
2	**12**
3	**16**
4	**18**

Chapter 9

Character Arrays and Strings

Introduction

- A string is a sequence of characters terminated by a null character ‘\0’.
- ‘\0’ is automatically encountered at the end of the string.

Declaration of string

char ch[6] ;

Here, ch is a string which can store a maximum of 5 characters and 1 character is reserved for ‘\0’.

Initialization of string

char ch[6] = “HELLO” ;

ch[0]	ch[1]	ch[2]	ch[3]	ch[4]	ch[5]
‘H’	‘E’	‘L’	‘L’	‘O’	‘\0’

char ch[6] = “Good” ;

ch[0]	ch[1]	ch[2]	ch[3]	ch[4]	ch[5]
‘G’	‘o’	‘o’	‘d’	‘\0’	‘\0’

char ch[6] = { ‘P’, ‘r’, ‘e’, ‘m’, ‘\0’ } ;

ch[0]	ch[1]	ch[2]	ch[3]	ch[4]	ch[5]
‘P’	‘r’	‘e’	‘m’	‘\0’	‘\0’

char ch[6] = “Program” ;

ch[0]	ch[1]	ch[2]	ch[3]	ch[4]	ch[5]
‘P’	‘r’	‘o’	‘g’	‘r’	‘a’

Initialization of string

Invalid Initialization

```
char str3[5];
str3 = “GOOD”;
```

I/O operations on strings

scanf()

```
char address[10] ;
scanf(“%s”, address);
```

printf()

```
char address[10] ;
scanf(“%s”, address);
printf(“%s”, address);
```

gets()

```
char address[10] ;
gets(address);
```

puts()

```
char address[10] ;
gets(address);
puts(address);
```

Reading Text till ~ is encountered

```
char line[10] ;
scanf("%[^~]", line);
```

I/O operations on strings

```
#include<stdio.h>
void main()
{
      char line[10] ;
      printf("Enter string\n");
      gets(line);
      printf("The string is: ");
      puts(line);
}
```

I/O operations on strings

```
#include<stdio.h>
void main()
{
      char a[10]="GLOBAL" ;
      int i,j;
      for(i=0;i<6;i++)
      {
            for(j=0;j<=i;j++)
            printf("%c\t", a[j]);
            printf("\n");
      }
}
```

```
#include<stdio.h>
void main()
{
      char a[10]="GLOBAL" ;
      int i,j;
      for(i=0;i<6;i++)
      {
            for(j=0;j<=i;j++)
            putchar(a[j]);
            printf("\n");
      }
}
```

Arithmetic Operations on Characters

To write a character in its integer representation, we may write it as an integer.

```
char x = ‘a’;
printf(“%d\n”, x);    // 97
x = ‘z’–1;
printf(“%d\n”, x);    // 122-1 = 121

char number[21] = “1988”;
int year = atoi(number);
```

The function atoi() converts the string “1988” (contained in number) to its numeric equivalent 1988 and assigns it to the integer variable year.

String conversion functions are stored in the header file <std.lib.h>.

String handling Functions

Function	Action
	#include<string.h>
strlen()	finds the length of a string excluding '\0'
strcpy()	copies one string over another
strcmp()	compares two strings
strcat()	concatenates two strings
strlwr()	Converts the string to lowercase letters
strupr()	Converts the string to uppercase letters

String handling Functions

strlen()

strcpy()

strcmp()

strcat()

strlwr()

strupr()

```
#include<string.h>
void main()
{
      char a[21] ;
      int len;
      printf("Enter a string\n");
      gets(a);
      len=strlen(a);
      printf("The length of the string is: %d", len);
}
```

String handling Functions

strlen()

strcpy()

strcmp()

strcat()

strlwr()

strupr()

works similar to string-assignment operator.

Syntax:

strcpy(string1, string2);

Assigns the contents of string2 to string1.
string2 may be a string variable or a string constant.

Example:

strcpy(city, “DELHI”);
will assign the string “DELHI” to the string variable city.

strcpy(city1, city2);
will assign the contents of the string variable city2 to the string variable city1.
The size of the array city1 should be large enough to receive the contents of city2.

String handling Functions

strlen()

strcpy()

strcmp()

strcat()

strlwr()

strupr()

```
#include<stdio.h>
#include<string.h>
void main()
{
      char a[21],b[21] ;
      printf("Enter a string\n");
      gets(a);
      strcpy(b,a);
      printf("The copied string is: %s",b);
}
```